

ViMo: Generating Motions from Casual Videos using Diffusion Models

Seminar Presentation

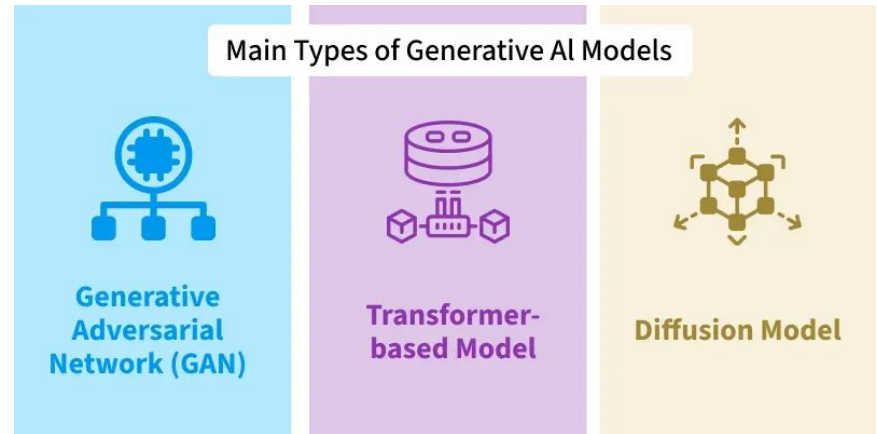
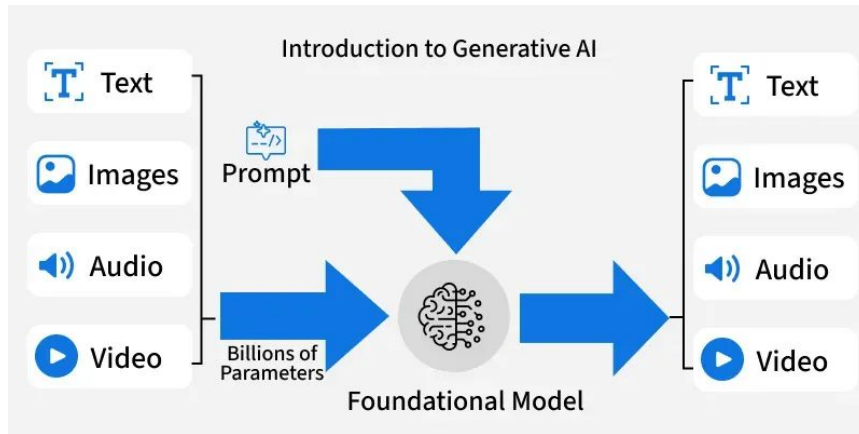
CS24M101 - A. Sai Jagadeesh

CS24M103 - Ram Krishna

CS24M106 - Logesh V

Generative AI (GenAI)

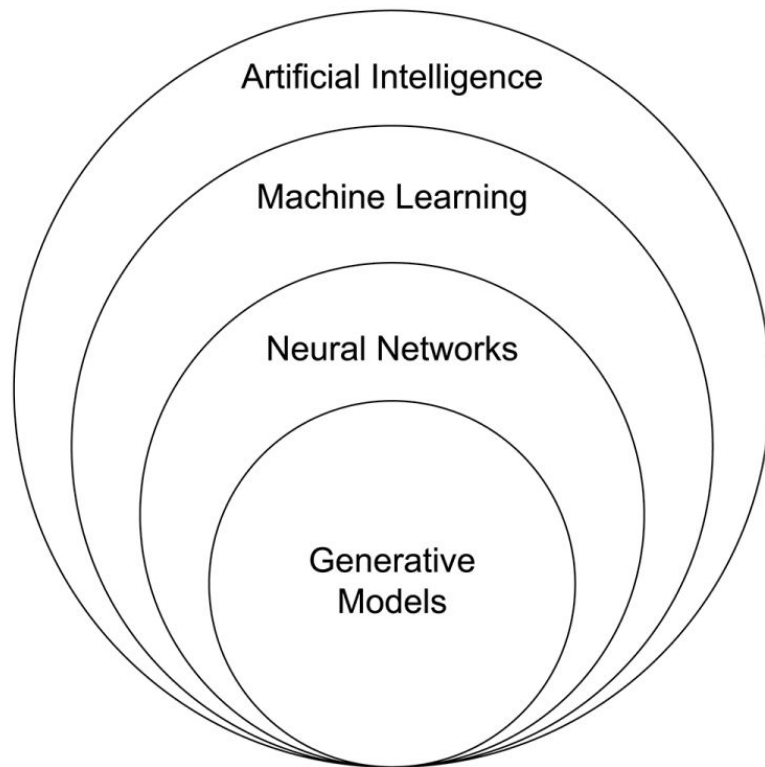
- Type of Artificial Intelligence that leverages AI to generate content or data
- Data can include text, images, audio, video, 3D models, code and video games
- Typically created in response to prompts (prompt engineering)
- Prompts are constructed inputs to language models to generate useful output
- Usually given with examples - zero shot versus few shot learning



How Do Generative AI Models Work?

How do they work? To create a generative model, an organization usually requires a large data set that is fed to the model in a process called training. The model learns the patterns in the data, adjusting its parameters to match the distribution of the training data. Following the training period, the model can generate new data that resembles the original data.

A generative model's significance lies in the high quality of its output and creativity – its ability to ascertain the patterns in data and create something new from its learnings – which has boundless applications across many fields. They particularly shine in tasks that require new content or augmentation of content such as creating human faces with a range of different emotions or generating a new art piece in the style of Monet.



Types of Generative AI Models

1. Transformers / Autoregressive Models

- Autoregressive Transformers Models generate sequences by predicting the next token based on all previous ones moving step by step through the text.
- The architecture relies on the transformer's self attention mechanism to capture context from the entire input so far making it highly effective for natural language and code generation.
- The autoregressive approach gives fine grained control over each output step but can be slower for long generations since tokens are generated one at a time.

2. Diffusion Models

- Diffusion models generate data such as images or audio by starting with pure random noise and gradually refining it into a coherent output through a series of denoising steps.
- Each step reverses a simulated diffusion process that added noise to real data during training.
- This iterative approach can produce highly detailed and realistic results specially in image synthesis
- Diffusion models are also versatile they can be adapted for inpainting, style transfer and conditional generation from text prompts.

3. Variational Autoencoders (VAEs) & Generative Adversarial Networks (GANs)

- VAEs and GANs were among the first deep learning architectures for generative tasks.
- A VAE encodes data into a compressed latent space and then decodes it back with a probabilistic twist that encourages smooth, continuous representations. This makes them good for controllable generation and interpolation between styles.
- GANs in contrast use two networks against each other a generator that tries to produce realistic outputs and a discriminator that tries to detect fakes.

4. Encoder Decoder Models

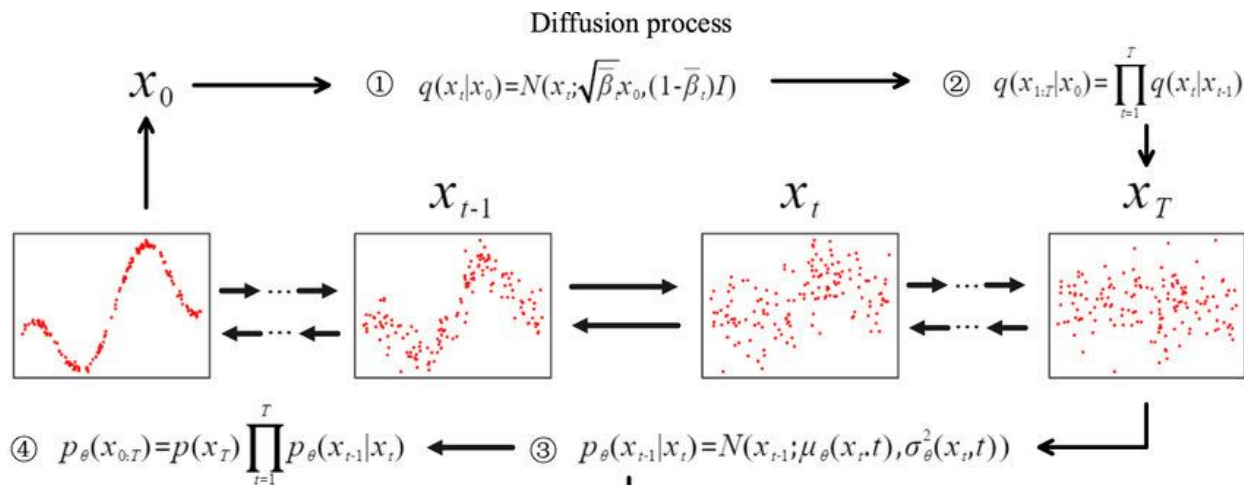
- Encoder decoder architectures consist of two stages: the encoder processes the input into a dense representation and the decoder generates the desired output from that representation.
- They are widely used for sequence to sequence tasks like language translation, summarization and image captioning.
- The encoder captures the full context of the input before the decoder starts producing tokens, allowing for strong performance on tasks that require global understanding rather than token by token prediction.

Denoising Diffusion Probabilistic Models (2020)

In training, diffusion models gradually diffuse a data point with random noise, step-by-step, until it's destroyed, then learn to reverse that diffusion process and reconstruct the original data distribution.

A trained diffusion model can then generate new data points that resemble the training data by simply denoising a random initial sample of pure noise. Conceptually, this is similar to a denoising autoencoder in which the noisy images act as latent variables.

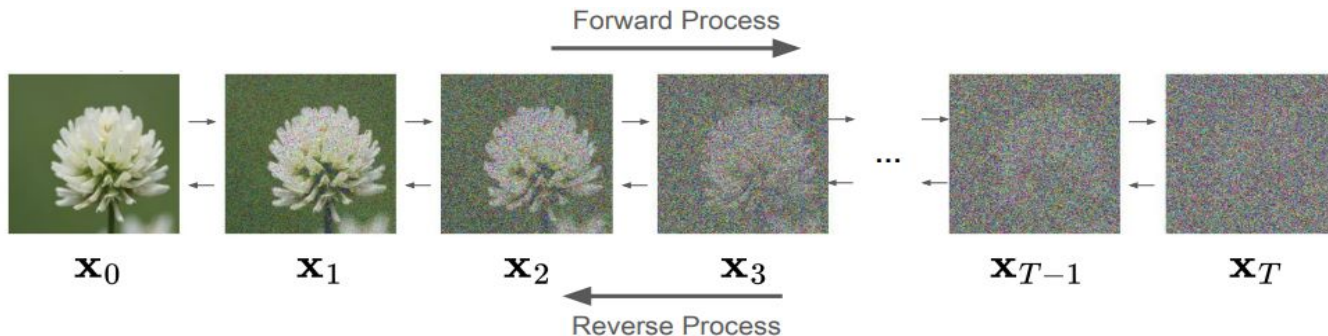
- Diffusion process involves gradually adding noise until data becomes pure noise.
- Learned to reverse a diffusion process to generate data



Directly transforming random noise into a coherent image is extremely difficult and complex, but transforming a noisy image into a slightly less noisy image is relatively easy and straightforward. Diffusion models therefore formulate the reverse diffusion process as an incremental, step-by-step transformation of a simple distribution (like Gaussian noise) to a more complex distribution (like a coherent image).

The process of training and then deploying a diffusion can be broken down into three key stages:

- The forward diffusion process, wherein an image from the training data set is transformed into pure noise—usually a gaussian distribution.
- The reverse diffusion process, wherein the model learns the inverse of each previous step in the original forward diffusion process.
- Image generation, wherein the trained model samples a random noise distribution and transforms it into a high-quality output by using the reverse diffusion process it has learned to denoise a random sample of gaussian noise.



Forward Process (Diffusion)

- Corrupting the data x_0 with noise over a sequence of T time steps.
- A Markov chain where each step slightly degrades the data by adding Gaussian noise.

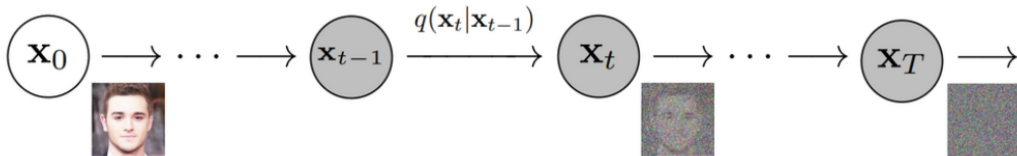


Figure: Forward Process.

$$q(x_t | x_{t-1}) = \mathcal{N}(x_t; \sqrt{1 - \beta_t} x_{t-1}, \beta_t I). \quad (1)$$

Forward Process (Diffusion)

Sampling x_t from conditional distribution $q(x_t | x_{t-1}) = \mathcal{N}(x_t; \sqrt{1 - \beta_t} x_{t-1}, \beta_t I)$,

$$x_t = \sqrt{1 - \beta_t} x_{t-1} + \sqrt{\beta_t} \epsilon. \quad (2)$$

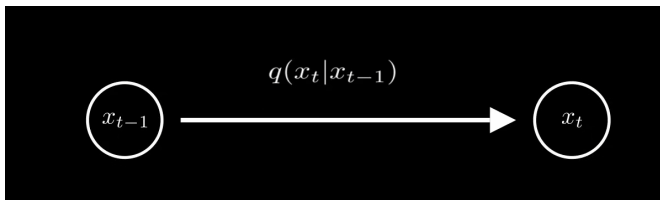
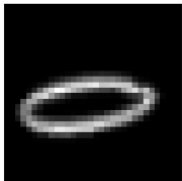


Figure: Forward Process (Markov chain).

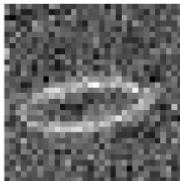
- where $\epsilon \sim \mathcal{N}(0, I)$ is Gaussian noise.
- β_t is the noise schedule that controls the amount of noise added at each step.

Forward Process (Diffusion)

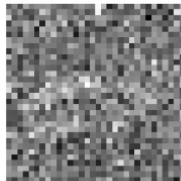
t = 0



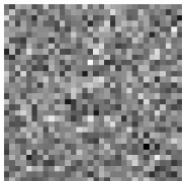
t = 100



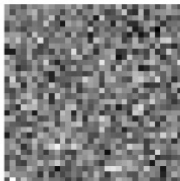
t = 200



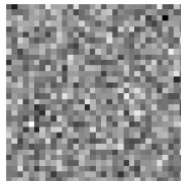
t = 300



t = 400



t = 500



Forward Process (Diffusion)

Reparameterization Trick:

$$q(x_t \mid x_{t-1}) = \mathcal{N}(x_t; \sqrt{1 - \beta_t} x_{t-1}, \beta_t I)$$

$$q(x_{t-1} \mid x_{t-2}) = \mathcal{N}(x_{t-1}; \sqrt{1 - \beta_{t-1}} x_{t-2}, \beta_{t-1} I)$$

$$q(x_t \mid x_{t-2}) = \mathcal{N}(x_t; \sqrt{(1 - \beta_t)(1 - \beta_{t-1})} x_{t-2}, [(1 - \beta_t)\beta_{t-1} + \beta_t] I)$$

$\vdots$

$$q(x_t \mid x_0) = \mathcal{N}(x_t; \sqrt{\bar{\alpha}_t} x_0, (1 - \bar{\alpha}_t) I)$$

$$\text{where } \bar{\alpha}_t = \prod_{s=1}^t (1 - \beta_s), \quad \alpha_t = 1 - \beta_t, \quad \text{and } \beta_t \in (0, 1), \quad t = 1, 2, \dots, T$$

x_t is the noisy data at time step t , and α_t controls the amount of noise added.

Forward Process (Diffusion)

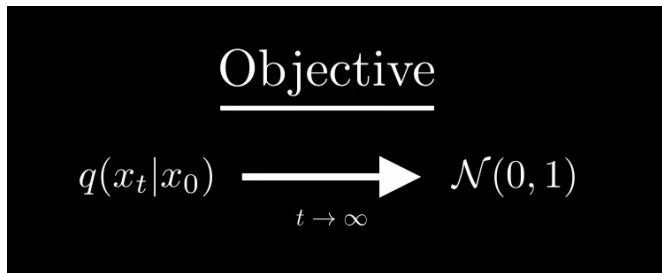


Figure: Forward Process.

$$q(x_t | x_0) = \mathcal{N}(x_t; \sqrt{\bar{\alpha}_t} x_0, (1 - \bar{\alpha}_t)I) \quad (3)$$

- As $t \rightarrow \infty$, $\bar{\alpha}_t \rightarrow 0$, because $\bar{\alpha}_t = \prod_{s=1}^t (1 - \beta_s)$,
- x_t becomes increasingly noisy until it approximates a Gaussian distribution $\mathcal{N}(0, I)$.

Reverse Process (Denoising)

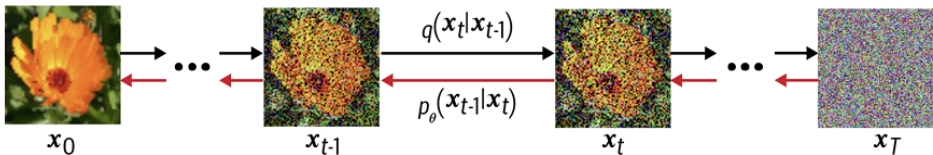


Figure: Reverse Process.

- Reconstructs the original data by denoising the noisy data in a series of steps reversing the forward diffusion.
- The reverse diffusion process is similar to the decoder in a VAE.
- In a VAE, the forward process (encoding images into noise) is learned / parameterized.
- In a Diffusion Model, the forward process is fixed / unparameterized.

Reverse Process (Denoising)

Modeled using a neural network to approximate the conditional probability $p_\theta(x_{t-1} \mid x_t)$

$$q(x_{t-1} \mid x_t, x_0) = \mathcal{N}\left(\mathbf{x}_{t-1}; \boldsymbol{\mu}_q(\mathbf{x}_t, \mathbf{x}_0), \sigma_q^2(t)\mathbf{I}\right), \quad (4)$$

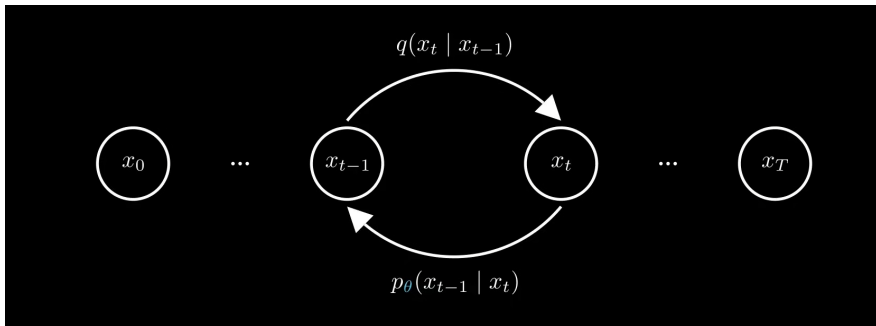


Figure: Reverse Process.

Reverse Process (Denoising)

(Proof in another slide)

where

$$\tilde{\mu}_t(x_t, x_0) = \frac{\sqrt{\bar{\alpha}_{t-1}} \beta_t}{1 - \bar{\alpha}_t} x_0 + \frac{\sqrt{\alpha_t} (1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t} x_t$$

and

$$\tilde{\beta}_t = \frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t} \beta_t$$

We know from the forward process:

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon$$

So,

$$x_0 = \frac{1}{\sqrt{\bar{\alpha}_t}} (x_t - \sqrt{1 - \bar{\alpha}_t} \epsilon)$$

Substituting, we get:

$$\tilde{\mu}_t(x_t, \epsilon) = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon \right). \quad (5)$$

Training Diffusion Models

Objective: We minimize the negative log-likelihood $-\log p_{\theta}(x_0)$.

Minimizing the objective function is equivalent to minimizing

$$\text{KL}(q(x_{t-1} \mid x_t, x_0) \parallel p_{\theta}(x_{t-1} \mid x_t))$$

so minimizing it gives $p_{\theta}(x_{t-1} \mid x_t) \approx q(x_{t-1} \mid x_t, x_0)$.

Algorithm 1 Training

```
1: repeat  
2:    $\mathbf{x}_0 \sim q(\mathbf{x}_0)$   
3:    $t \sim \text{Uniform}(\{1, \dots, T\})$   
4:    $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$   
5:   Take gradient descent step on  
        $\nabla_{\theta} \left\| \epsilon - \epsilon_{\theta}(\sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, t) \right\|^2$   
6: until converged
```

Figure: Training Algorithm.

Sampling from Diffusion Models

Algorithm 2 Sampling

```
1:  $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ 
2: for  $t = T, \dots, 1$  do
3:    $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$  if  $t > 1$ , else  $\mathbf{z} = \mathbf{0}$ 
4:    $\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left( \mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}} \epsilon_{\theta}(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}$ 
5: end for
6: return  $\mathbf{x}_0$ 
```

Figure: Sampling Algorithm.

Conditional Diffusion Models

- Extend standard diffusion models to **condition on extra information** y (e.g., class label, text, image).
- Goal: model $p_\theta(x_0 \mid y)$ instead of unconditional $p_\theta(x_0)$.
- Training uses **same denoising objective** as unconditional DDPM:

$$\mathbb{E}_{x_0, t, \epsilon} \left[\|\epsilon - \epsilon_\theta(x_t, t, y)\|^2 \right]. \quad (6)$$

- During sampling, model generates data consistent with condition y .
- Widely used in **text-to-image, class-conditional image generation, super-resolution, and inpainting**.
- Provides **fine-grained control** over generation while maintaining high sample quality.

Popular Applications of Diffusion Models

Text-to-Image Generation

- DALL-E 2 (OpenAI)
- Imagen (Google)

Diffusion Models Outperform GANs and VAEs

- **Stable Training:** No adversarial loss; avoids mode collapse and training instability common in GANs.
- **High-Quality & Diverse Samples:** VAE does one-shot generation whereas diffusion models iteratively denoise over T timesteps..
- **Exact Likelihood Estimation:** Allows likelihood-based training, enabling better probabilistic modeling and principled evaluation.

Introduction

- Realistic 3D human motions are fundamental for bringing virtual characters to life in applications like animation, gaming, and the metaverse, but the traditional creation of such motions is labor-intensive and costly, relying on mocap systems or skilled animators.
- While recent data-driven methods based on generative models (diffusion, VAE, GANs) have improved motion synthesis, they are constrained by the limited diversity and scalability of motion capture datasets available for training, resulting in less flexible and less diverse generated motions.
- Existing methods often require matched ground-truth pairs (e.g., audio-to-motion, text-to-motion), tying their output to narrow domains and making them unable to generalize or scale to the full diversity found in natural human actions.
- In contrast, abundant and diverse human activities exist in everyday videos online, but extracting realistic 3D motions from these “casual” videos is extremely challenging due to unknown camera movements, frequent occlusions, and complex editing, which defeat traditional pose estimation and motion reconstruction pipelines.
- Learning 3D motions from such unconstrained videos is desirable for unlocking broad application potential and dataset construction, but remains a formidable challenge due to the intricacies of real-world video content and the near-impossibility of recovering exact camera parameters.

Motivation

Have proposed a novel video-to-motion generation model by taking videos as conditions to generate various motions and showcase its great potential in enabling three important downstream applications:

1. Firstly, it could be an efficient way to produce large-scale motion datasets. This new dataset could be used to facilitate data-driven tasks such as music-to dancing generation, motion recognition and prediction.
2. Secondly, our model could power up few-shot stylization in the dancing generation. Specifically, we could leverage the music-to-motion generation pipeline to generate a specific style of dance according to the pose style offered by a few video examples.
3. Thirdly, we could form a new video-guided motion completion and editing task which could fill in the missing part of the motion by the similar content defined by any source videos.



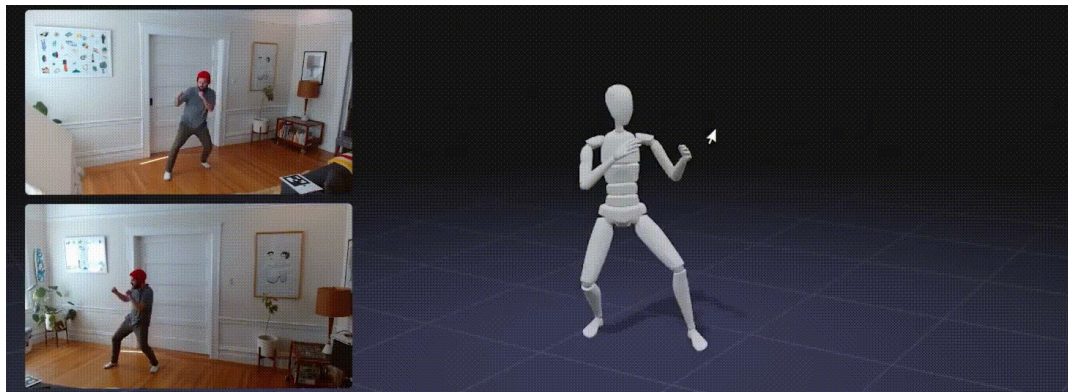
Related Work

- **Human Motion Generation Methods:** Prior works generate 3D human motions using conditional generative models—such as diffusion models, VAEs, and GANs—that take various signals (noise, text, music, or scene context) as input. These approaches have mainly focused on generating motions within the categories and domains represented in their mocap-captured training data.
- **Limitations of Existing Approaches:** Action-conditioned and signal-conditioned models are fundamentally limited by dataset scope, only synthesizing motions seen in the training set and requiring accurate pairing between input conditions and ground truth motions. Diversity and flexibility in generated motions remain a major challenge.
- **Video-to-Motion and 3D Pose Reconstruction:** Previous efforts attempted to reconstruct 3D human motions from video using pose estimation, either by regressing joint positions from single images or lifting 2D keypoints to 3D using deep learning models. However, state-of-the-art techniques (like MotionBERT) work well only when camera parameters are known or fixed—complex camera movement or occlusions in real-world videos drastically reduce their reliability.
- **Challenges in Real-World Video:** Most prior video-based approaches break down when facing the complexities of “casual” videos—unknown or moving cameras, frequent occlusions, or montage editing—causing pose estimates to be inconsistent and difficult to string into smooth, plausible motion sequences.
- **ViMo’s Contribution:** The paper identifies under-explored potential in leveraging the abundance of casual videos for 3D motion generation and proposes a new approach to robustly generate diverse, realistic motions directly from noisy video-derived 2D poses, even in challenging camera scenarios where traditional regression-based pipelines fail.

Video-to-Motion: Relevance and Generative Bridges

The Video-to-Motion problem stands at the intersection of computer vision, graphics, and generative AI, offering transformative potential for applications ranging from animation and gaming to virtual reality and digital avatars. This challenge is highly relevant because it enables scalable, low-cost creation of realistic human movements directly from ubiquitous online video sources, bypassing traditional bottlenecks like expensive motion capture hardware, manual annotation, or heavily curated datasets.

Generative models are increasingly bridging the gap between noisy video-derived poses and usable motion sequences by refining estimates, incorporating physical plausibility, and synthesizing diverse, temporally-coherent 3D actions. These advances unlock the use of video as a rich, flexible data resource, open new possibilities for animating characters, and make motion synthesis broadly accessible for interactive and creative platforms.



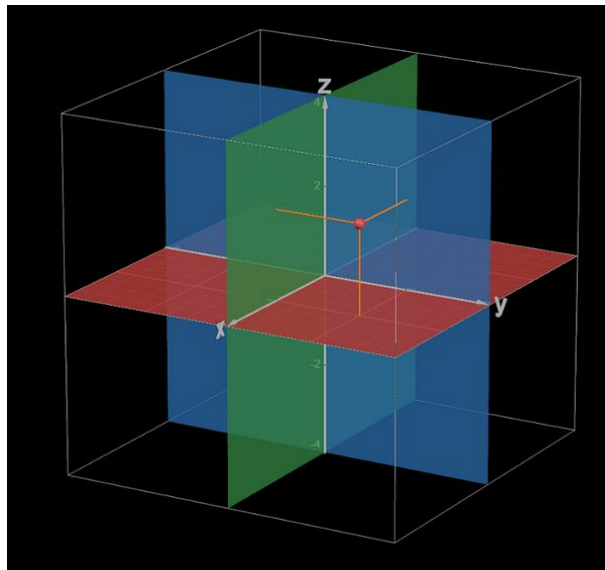
Introduction to 3D Representations

3D representations describe how human body motion, skeleton rotations, and object orientations are expressed mathematically and encoded for computational processing. These representations are fundamental across computer vision, animation, robotics, and motion synthesis, as they enable accurate modeling and prediction of spatial orientation, rotation dynamics, and temporal transitions.

Goal: Accurately model orientation, rotation, and position in 3D space.

Common forms:

- Euler Angles
- Axis-Angle Representation
- Quaternions
- Rotation Matrices
- 6D Rotation Representation (used in ViMo)



Local vs Global Coordinate Systems

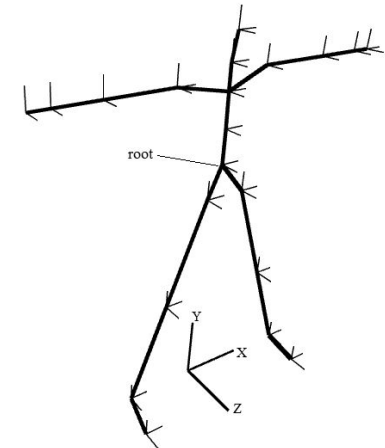
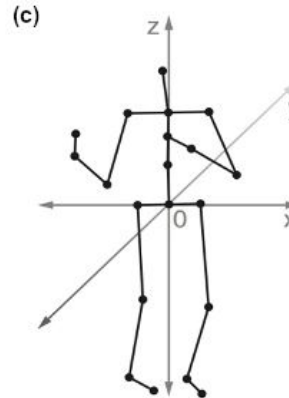
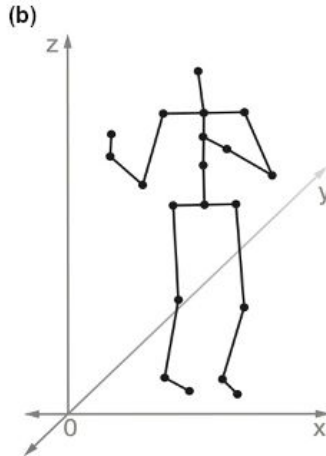
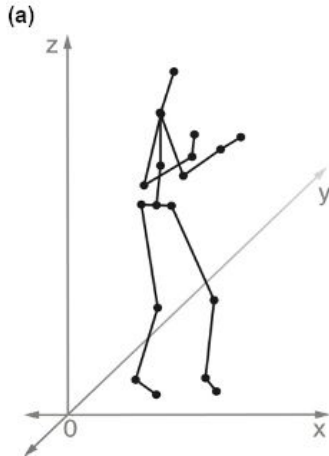
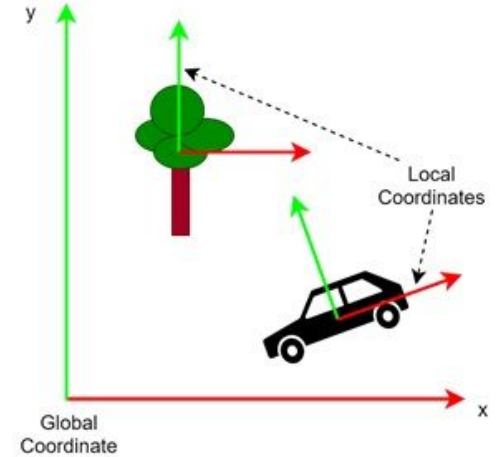
Local Coordinate System:

- Each joint's rotation is defined relative to its parent joint.
- Maintains skeletal hierarchy.
- Used in **Forward Kinematics (FK)** to compute global positions.

Global Coordinate System:

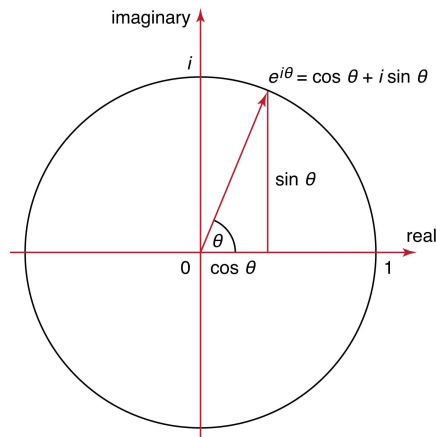
- Represents the position and orientation of each joint in the world space.
- Used for visualization and physical simulation.

Relation: Global = Parent(Global) $\times$ Local Rotation.



Euler Angles (Common)

- Represent 3D rotation as three sequential rotations around X, Y, Z axes.
- Common in simple animation pipelines.
- **Example:** $R = R_z(\theta_z) * R_y(\theta_y) * R_x(\theta_x)$
- **Advantages:** Intuitive; matches real-world pitch, yaw, roll for human understanding



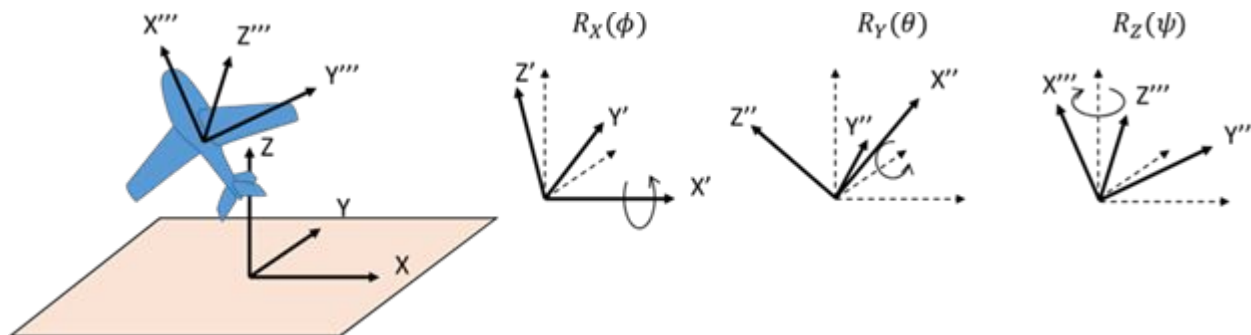
© Encyclopædia Britannica, Inc.

Euler's Formula as a Matrix

$$e^{ix} = \cos x + i \sin x$$

$$= \begin{bmatrix} \cos x & -\sin x \\ \sin x & \cos x \end{bmatrix}$$

$$e^{i\pi} = -1 = \begin{bmatrix} -1 & 0 \\ 0 & -1 \end{bmatrix}$$



$$R = R_z(\psi)R_x(\phi)R_y(\theta)$$

$$= \begin{bmatrix} \cos \psi & -\sin \psi & 0 \\ \sin \psi & \cos \psi & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \phi & -\sin \phi \\ 0 & \sin \phi & \cos \phi \end{bmatrix} \begin{bmatrix} \cos \theta & 0 & \sin \theta \\ 0 & 1 & 0 \\ -\sin \theta & 0 & \cos \theta \end{bmatrix}$$

$$= \begin{bmatrix} \cos \psi \cos \theta - \sin \phi \sin \psi \sin \theta & -\sin \psi \cos \phi & \sin \theta \cos \psi + \sin \phi \sin \psi \cos \theta \\ \sin \psi \cos \theta + \sin \phi \sin \theta \cos \psi & \cos \phi \cos \psi & \sin \theta \sin \psi - \sin \phi \cos \psi \cos \theta \\ -\sin \theta \cos \phi & \sin \phi & \cos \phi \cos \theta \end{bmatrix}$$

Axis-Angle Representation

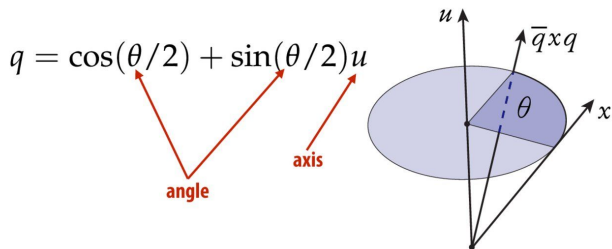
It uses just two pieces of information:

- a direction vector (which way to spin) and
- an angle (how much to spin).

Advantage: simplicity—it's geometrically easy to understand.

Drawback: discontinuity: smooth transition rotations behaves erratically at certain angles, making it unsuitable for neural networks trying to learn smooth motion sequences.

■ Given axis u , angle θ , quaternion q representing rotation is



■ Much easier to remember (and manipulate) than matrix!

$$\begin{bmatrix} \cos \theta + u_x^2 (1 - \cos \theta) & u_x u_y (1 - \cos \theta) - u_z \sin \theta & u_x u_z (1 - \cos \theta) + u_y \sin \theta \\ u_y u_x (1 - \cos \theta) + u_z \sin \theta & \cos \theta + u_y^2 (1 - \cos \theta) & u_y u_z (1 - \cos \theta) - u_x \sin \theta \\ u_z u_x (1 - \cos \theta) - u_y \sin \theta & u_z u_y (1 - \cos \theta) + u_x \sin \theta & \cos \theta + u_z^2 (1 - \cos \theta) \end{bmatrix}$$

Quaternions

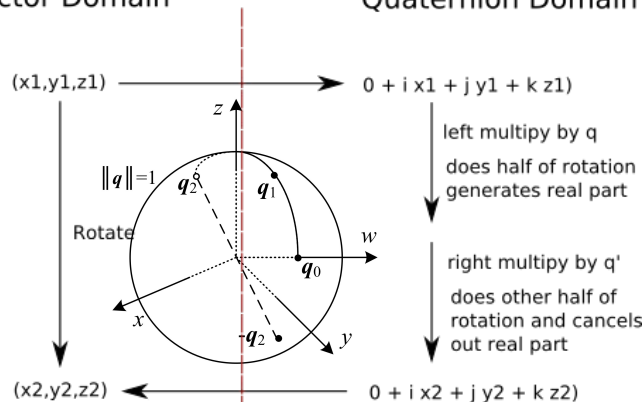
The power of quaternions lies in their mathematical smoothness—when you smoothly blend between two quaternions (called interpolation), the rotation transitions naturally without flipping or singularities.

Hidden constraint: they must satisfy a special property called unit norm ($\|q\| = 1$)

constraint maintenance burden: During training, without explicit enforcement of this constraint, a quaternion can "drift" away from validity, losing its rotational meaning

Vector Domain

Quaternion Domain



Rotation Matrices

- A rotation matrix $R \in \mathbb{R}^{(3 \times 3)}$ is a square matrix that describes how a 3D object or joint rotates in space. It is defined by two critical mathematical properties: Orthonormality and Determinant = 1.

For any n -dimensional rotation matrix R acting on $\mathbb{R}^n$,

$$R^T = R^{-1}$$

(The rotation is an [orthogonal matrix](#))

$$\det R = \pm 1$$

$$R = \begin{bmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \end{bmatrix}$$

- **Properties:**

- Orthogonal: $R^T R = I$

- **Advantages:** Rotation matrices are geometrically intuitive and computationally straightforward—applying a rotation is simply a matrix-vector multiplication. They explicitly encode the full spatial orientation.
- **Disadvantages:** Rotation matrices are over-parametrized—they use 9 numbers to represent only 3 deg of freedom (the dim of the 3D rotation group $SO(3)$). If the network directly predicts all 9 elements, there is no guarantee the output will satisfy the orthonormality and determinant constraints. Post-processing steps like QR decomposition or Gram-Schmidt orthonormalization are needed to project the output onto the valid rotation manifold, but these operations are not directly differentiable or optimized through the network, leading to suboptimal learning.

6D Rotation Representation

The 6D rotation representation is an elegant solution that eliminates the constraints of rotation matrices while maintaining their geometric validity. It was introduced specifically to provide a continuous, constraint-free parameterization suitable for neural network regression tasks.

Rather than predicting all 9 elements of R , the 6D representation stores only the first two columns of the rotation matrix, denoted $\mathbf{a}_1$ and $\mathbf{a}_2$, each a 3D vector. This gives 6 parameters total. The third column $\mathbf{a}_3$ is then deterministically reconstructed using the cross product:

$$\mathbf{a}_3 = \mathbf{a}_1 \times \mathbf{a}_2$$

Why This Works: The cross product and normalization steps are differentiable operations, so gradients flow smoothly through them during backpropagation. The network can predict arbitrary 6D vectors in unconstrained Euclidean space, and the reconstruction procedure guarantees a valid rotation matrix by design

To ensure orthonormality, the predicted vectors are normalized and orthogonalized via the Gram-Schmidt process:

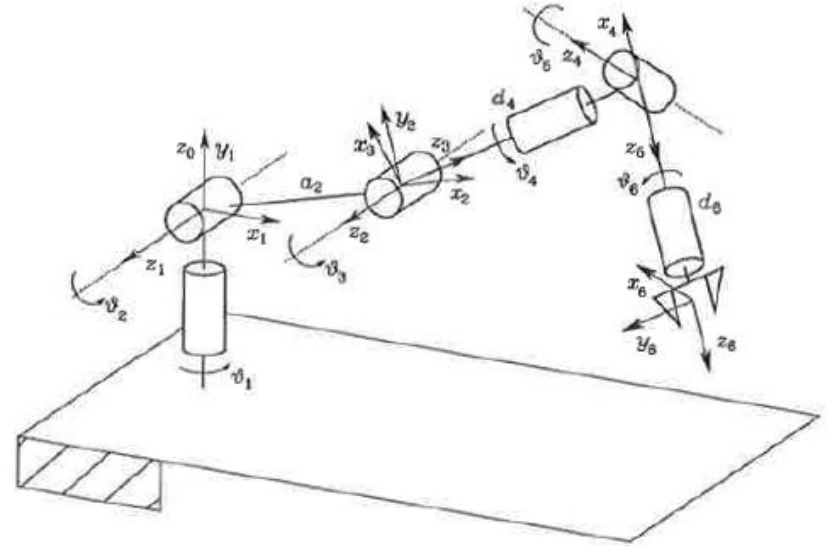
- Normalize the first vector: $\mathbf{b}_1 = \frac{\mathbf{a}_1}{\|\mathbf{a}_1\|}$
- Orthogonalize and normalize the second: $\mathbf{b}_2 = \frac{\mathbf{a}_2 - (\mathbf{a}_2 \cdot \mathbf{b}_1)\mathbf{b}_1}{\|\mathbf{a}_2 - (\mathbf{a}_2 \cdot \mathbf{b}_1)\mathbf{b}_1\|}$
- Compute the third via cross product: $\mathbf{b}_3 = \mathbf{b}_1 \times \mathbf{b}_2$

The final rotation matrix is $\mathbf{R} = [\mathbf{b}_1 \mid \mathbf{b}_2 \mid \mathbf{b}_3]$

Forward Kinematics

Forward kinematics is a fundamental concept in robotics, computer animation, and motion synthesis that describes how to compute the position of various parts (like hands or feet) of a moving body, given all the joint angles or rotations. In simple terms, if you know how every joint in an arm (shoulder, elbow, wrist) is rotated, you can mathematically calculate exactly where the hand will be in three-dimensional space.

This process starts from a fixed part (often called the "root" or base, like the pelvis or shoulder), and step by step applies each joint's rotation or extension to figure out where the next part is, eventually reaching the endpoint (like the fingertip or toe). In animation and motion models like ViMo, forward kinematics converts predicted joint rotations (such as those encoded by 6D representations) into actual 3D positions for the skeleton, ensuring the overall motion appears physically plausible and coordinated.



Mathematical Formulation

Forward Kinematics computes the **global 3D positions** of each joint in a skeleton, given the **local rotations** (and translations) of joints and the skeleton's hierarchy.

Input: Local joint rotations + bone offsets

Output: Global joint positions in world coordinates.

If a joint j has a parent $p(j)$:

$$G_j = G_{p(j)} \times \begin{bmatrix} R_j & t_j \\ 0 & 1 \end{bmatrix}$$

where:

- G_j : global transformation of joint j
- R_j : local rotation matrix of joint j
- t_j : translation from parent to joint j (bone vector)
- $G_{p(j)}$: parent's global transform

The **root joint** (usually pelvis) starts with:

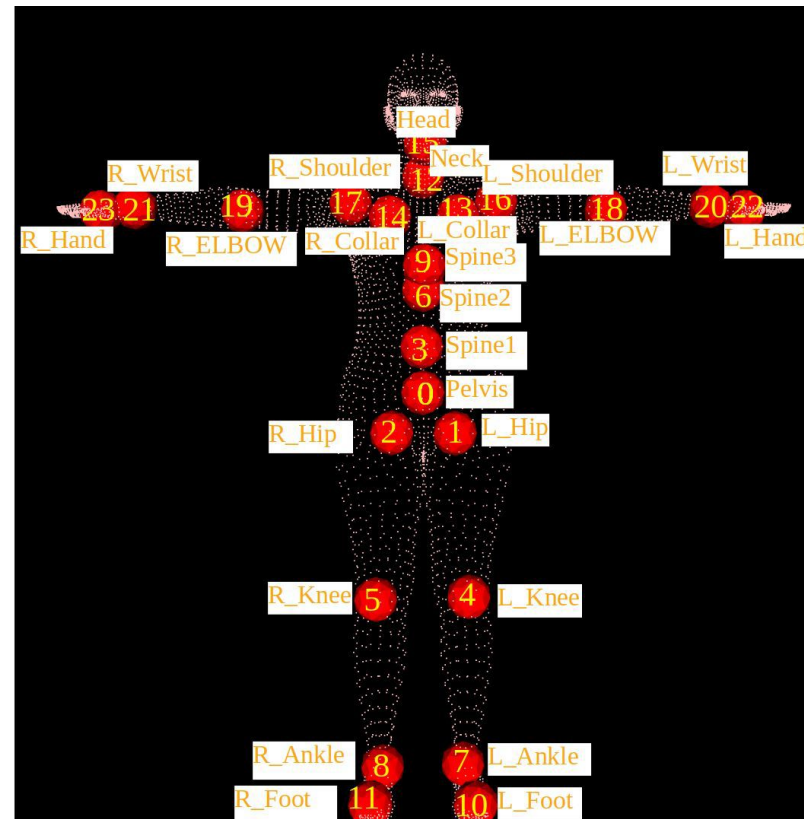
$$G_{root} = \begin{bmatrix} R_{root} & t_{root} \\ 0 & 1 \end{bmatrix}$$

$$T = \left[\begin{array}{ccc|c} & R & & T \\ & & & \\ & & & \\ \hline 0 & 0 & 0 & 1 \end{array} \right] = \left[\begin{array}{ccc|c} \cos \theta & -\sin \theta \cos \alpha & \sin \theta \sin \alpha & r \cos \theta \\ \sin \theta & \cos \theta \cos \alpha & -\cos \theta \sin \alpha & r \sin \theta \\ 0 & \sin \alpha & \cos \alpha & d \\ \hline 0 & 0 & 0 & 1 \end{array} \right]$$

SMPL (Tool used for FK in implementation)

'Pelvis',	# 0
'L_Hip',	# 1
'R_Hip',	# 2
'Spine1',	# 3
'L_Knee',	# 4
'R_Knee',	# 5
'Spine2',	# 6
'L_Ankle',	# 7
'R_Ankle',	# 8
'Spine3',	# 9
'L_Foot',	# 10
'R_Foot',	# 11

'Neck',	# 12
'L_Collar',	# 13
'R_Collar',	# 14
'Head',	# 15
'L_Shoulder',	# 16
'R_Shoulder',	# 17
'L_Elbow',	# 18
'R_Elbow',	# 19
'L_Wrist',	# 20
'R_Wrist',	# 21
'L_Hand',	# 22
'R_Hand',	# 23



References:

1. [SMPL Format] (<https://medium.com/@bhipanshudhupar/what-is-smpl-the-3d-human-body-model-powering-modern-ai-and-animation-c654a0284800>)
2. [SMPL made simple FAQs] (<https://files.is.tue.mpg.de/black/talks/SMPL-made-simple-FAQs.pdf>)

Problem Definition

Given / Assumptions (for simplicity);

- input casual video only contains a single person is to be considered
- basic 2D pose estimation results are obtained from existing pose extractors such as OpenPose and AlphaPose vision models
- The 2D pose sequences are denoted as $p \in \text{shape}(S, J_{2d}, C_{in})$ where $S=150$ (i.e. 5 sec of 30fps video), each 2D pose skeleton follows the standard COCO format $\rightarrow$ 17 joints, $C_{in}=3$ (x,y, conf)
- missing joints and low-confidence joints will be filtered out

Objective:

- To Generate 3D motion $M \in \text{shape}(S, J_{3d} \cdot C + 4 + 3) \Rightarrow [\text{Rot}, \text{FC}, \text{Rp}]$, where:
 - Rot = flattened ver { seq of joints rotations $\in \text{shape}(J_{3d}=24, C=6)$ in 6D rot space,
 - FC = foot contact labels $\in \text{shape}(4) \rightarrow$ (ankles and feet joints)*(left and right), 1 $\Rightarrow$ its touching ground.
 - Rp = root position $\in \text{shape}(3)$, i.e. (x, y, z)

AIST++ Dataset

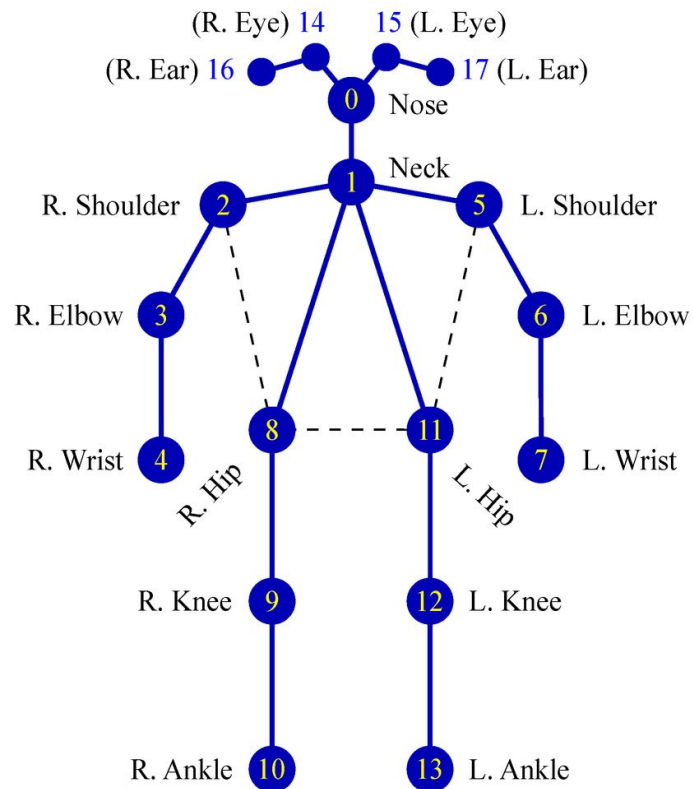
- ❖ Each sequence name corresponds with a set of multi-view video names in the AIST Dance Video DB.

Human Motion Sequence:

- Each SMPL-format human motion sequence is stored in a .pkl file with the following attributes:
- 'smpl_poses': Sequences of SMPL pose parameters. Array shape is (N, 24, 3).
- 'smpl_trans': Motion 3D trajectory. Array shape is (N, 3).

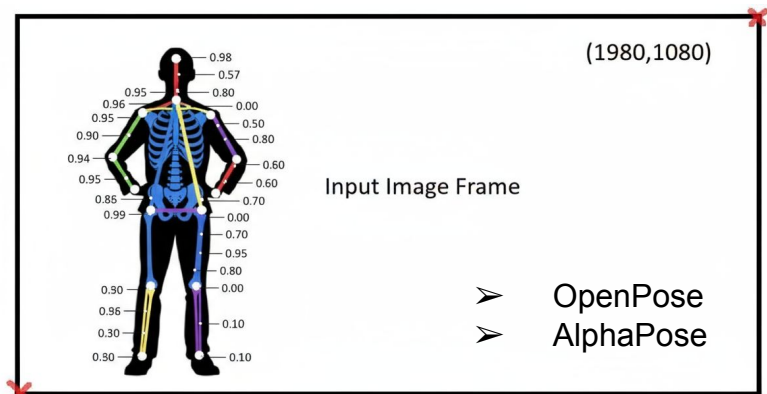
Keypoints2d Annotation: Multi-view frame-by-frame 2D keypoints detection results. Array shape is (9, N, 17, 3) where

- The first dim represents Individual environment settings, each with 9 cameras.
- J2D = 17 (no.of joints in 2d repr. w.r.t coco semantics).
- The last dim, i.e. each joint contains (x, y, confidence).
- **NOTE:** keypoints are given directly in image pixel coordinates, not normalized or made relative to a smaller scale. The AIST videos are 1080p resolution, with frames sized at 1920×1080 pixels — the keypoints are not scaled.



COCO 17 keypoint order

Pose Estimators



(0,0) (x,y) => Pixel Coordinates
Confidence => Accuracy Score (0-1)



2D Pose Estimation on different poses.

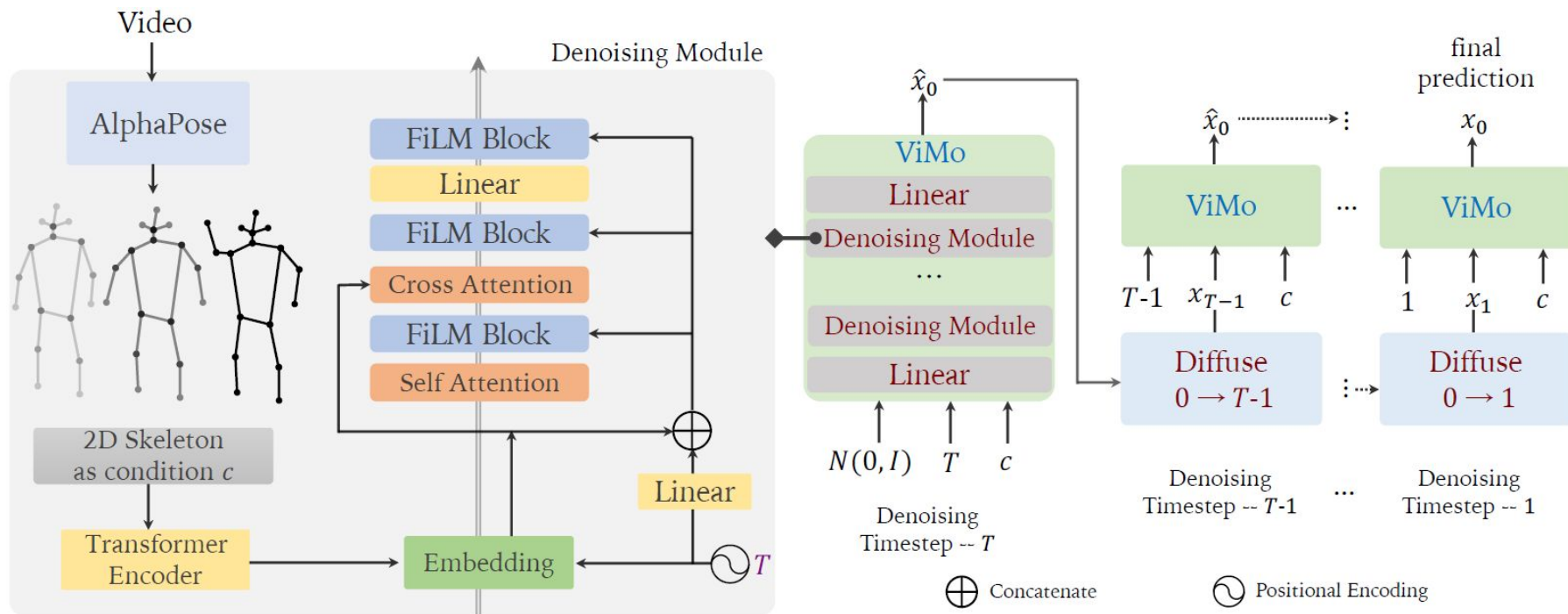
Preprocessing dataset for ViMo training

- ❖ Resamples 60 → 30 FPS by taking every 2nd frame (paper re-aligns to 30 FPS for training).
- ❖ Cuts sequences into 5s clips ($S = 150$ frames).
- ❖ Keeps cond_2d as views $x 150 \times 17 \times 3$ (multi-view 2D detections).
- ❖ Builds m3d_gt per clip as 150×151 (24×6 rotations + 4 foot contact + 3 root).
- ❖ Skips sequences listed in ignore_list.txt if present.

- ❖ How do I convert SMPL part rotations to other formats:
 - [pytorch3d](<https://github.com/facebookresearch/pytorch3d>)
 - refer page 32 at "SMPL made simple FAQ"

Foot contacts are calculated by checking the vertical velocity of each ankle and foot joint in consecutive frames; if the velocity is below a small threshold, that joint is marked as being in contact with the ground. This results in a binary contact label for each of four foot joints (left/right ankle and foot) per frame, allowing the detection of ground contact moments throughout the sequence.

ViMo Pipeline



ViMo Framework

Considering the generation as a **Markov noising Process** following [18], the 2D pose sequence p would join the noising process to finally produce a 3D motion $m \in \mathbb{R}^{S \times (24 \times 6 + 4 + 3)}$ corresponding to the conditional signal.

A typical diffusion process is employed as an initial step. The noising process satisfies

$$q(m_t^{1:S} | m_{t-1}^{1:S}) = \mathcal{N}(\sqrt{\alpha_t} m_{t-1}^{1:S}, (1 - \alpha_t) I), \quad (1)$$

with latent $\{m_t^{1:S}\}_{t=0}^T$ which is assumed as Gaussian distribution and $x_0^{1:S}$ is sampled from original data $m \sim q(M | p)$. $\alpha_t \in (0, 1)$ is the constant parameter at noising step $t \in [0, 1, \dots, T]$.

The reversed denoising process is modelled with a neural network starting from the Gaussian distribution $m_T^{1:S} \sim \mathcal{N}(0, I)$ with simple objective [18]

$$\hat{m}_0 = D(m_t, t, p),$$

where $\hat{m}_0$ obeys the original data distribution $q(M | p)$. It is used for computing the posterior distribution $q(m_{t-1} | m_t, m_0, p)$ at denoising step t . It needs to point out that result motions are directly predicted rather than reparameterization of vanilla epsilon.

- a transformer based diffusion model is leveraged to align the synthesis motion to the 2D pose sequences.
- [18] → ref here the DDPM paper published at 2020 explained earlier

Note: sampling processing is employed during training in a *classifier-free manner and Classifier-free diffusion guidance*.

The resulting estimate is composed of 25% unconditioned guidance and 75% conditioned guidance, balancing between diversity and fidelity by allowing a 25% probability to drop off the guidance.

Loss Functions

$$\mathcal{L}_{\text{simple}} = E_{m \sim q(\mathbf{M}|p), t \sim [1, N]} \left[\|m_0 - D(m_t, t, c)\|_2^2 \right]$$

$$\mathcal{L}_{\text{joints}} = \frac{1}{S} \sum_{i=1}^S \left\| FK(R_0^i) - FK(\hat{R}_0^i) \right\|_2^2,$$

$$\mathcal{L}_{\text{vel}} = \frac{1}{S-1} \sum_{i=1}^{S-1} \left\| (R_0^{i+1} - R_0^i) - (\hat{R}_0^{i+1} - \hat{R}_0^i) \right\|_2^2,$$

$$\mathcal{L}_{\text{foot}} = \frac{1}{S-1} \sum_{i=1}^{S-1} \left\| \left(FK(\hat{R}_0^{i+1}) - FK(\hat{R}_0^i) \right) \cdot \hat{f}_i \right\|_2^2.$$

$$\mathcal{L} = \underbrace{\mathcal{L}_{\text{simple}}}_{\text{diffusion}} + \underbrace{\lambda_1 \mathcal{L}_{\text{joints}} + \lambda_2 \mathcal{L}_{\text{vel}} + \lambda_3 \mathcal{L}_{\text{foot}}}_{\text{auxiliary}}$$

Applications

3D Motion Dataset Construction:

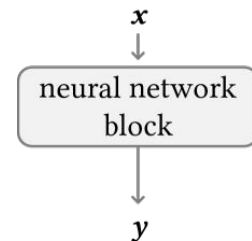
- **Key Accomplishment:** Collected 52 Chinese classic dancing competition videos and generated 750 high-quality 3D motion clips spanning 63 minutes of motion data.
- **Downstream Benefits:** This dataset enables downstream tasks such as music-to-motion generation, motion recognition, motion prediction, and expands dance genre representation (Chinese classical dance was previously absent from major datasets).
- **Scalability:** The framework demonstrates potential to expand datasets by 5x to 50x by leveraging the abundance of freely available video content online.

Few-Shot Dancing Stylization:

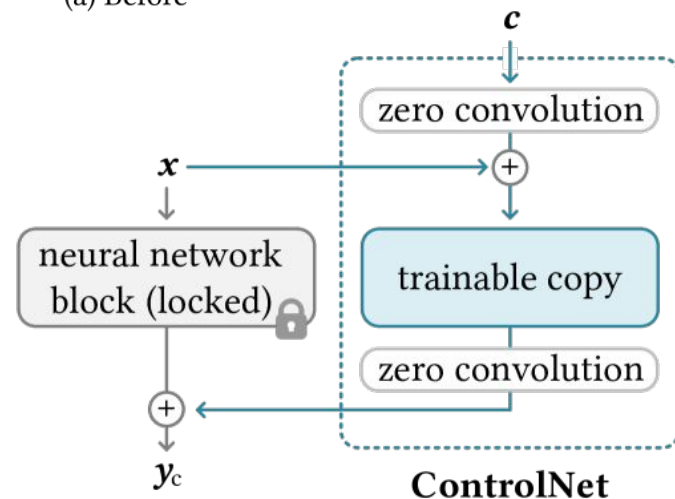
- **Paradigm Shift:** Overcomes the constraint of existing music-to-dance models (e.g., EDGE) that are limited to predefined categories (10 street dance categories in AIST dataset).
- **Technical Innovation:** Uses zero-convolution blocks for parameter-efficient fine-tuning—keeps the pre-trained music-to-motion knowledge frozen while adaptively learning new style distributions, preventing catastrophic forgetting.
- **User Study Results:** Zero-convolution adaptation showed 56–66% win rate versus direct fine-tuning (34–44%), demonstrating significantly better style transfer quality.

Video-guided Motion Completion

- Enables three essential editing tasks—
 - a. **motion in-betweening** (filling intermediate frames between keyframes),
 - b. **in-filling** (completing missing motion segments), and
 - c. **blending** (seamlessly transitioning between different motion styles).
- **Efficiency:** Provided only partial keyframes or reference clips; the model automatically generates plausible, smooth intermediate motions, significantly reducing manual work.
- **Technical Approach:** Leverages diffusion inpainting techniques—applies a binary mask during denoising to preserve constraint regions while generating natural transitions in unconstrained frames.



(a) Before



(b) After

**An Illustration of
Zero-Conv Blocks**

Metrics - 1. Fréchet Inception Distance (FID)

FID measures the statistical similarity between two sets of images (or motion feature distributions): one from real data and one from generated data. In ViMo's context, it compares generated 3D motion sequences with ground-truth motions from the AIST++ dataset to assess motion quality.

FID models both the real and generated motion feature distributions as multivariate Gaussian distributions: $\mathcal{N}(\mu, \Sigma)$ and $\mathcal{N}(\mu', \Sigma')$.

The distance between these two Gaussian distributions is computed using the 2-Wasserstein distance (also called Fréchet distance):

$$\text{FID}^2 = \|\mu - \mu'\|_2^2 + \text{tr} \left(\Sigma + \Sigma' - 2(\Sigma\Sigma')^{1/2} \right)$$

where:

- μ, μ' are the mean feature vectors of real and generated motion distributions
- Σ, Σ' are the covariance matrices
- $\|\mu - \mu'\|_2^2$ measures the Euclidean distance between means (how far apart the "centers" of the two distributions are)
- $\text{tr}(\cdot)$ denotes the matrix trace (sum of diagonal elements)
- $(\Sigma\Sigma')^{1/2}$ is the matrix square root of the product of covariances

Metrics - 2. Diversity Metrics

Diversity metrics quantify how varied or different the generated motions are from each other. High diversity means the model can produce a wide range of distinct movements rather than repetitive or mode-collapsed outputs.

Diversity is computed as the mean pairwise distance between motion feature vectors across all generated samples:

$$\text{Div} = \frac{1}{N(N-1)} \sum_{i=1}^N \sum_{j \neq i} d(\mathbf{f}_i, \mathbf{f}_j)$$

where:

- N is the total number of generated motion samples
- $\mathbf{f}_i, \mathbf{f}_j$ are feature vectors (kinetic or geometric) extracted from motions i and j
- $d(\cdot, \cdot)$ is a distance function, typically Euclidean distance $\|\mathbf{f}_i - \mathbf{f}_j\|_2$

ViMo computes two variants:

- **Div_k**: Diversity using kinetic features
- **Div_m**: Diversity using geometric features

Interpretation: Higher diversity scores indicate greater variety in generated motions. A model with low diversity might be generating similar-looking dances repeatedly (mode collapse).

Metrics - 3. Beat Align Score (BA)

BA measures how well the generated dance motion synchronizes with the rhythm and beats of the input music. This is critical for music-to-dance tasks where movements should align with musical timing.

The BA score computes the average temporal offset between each music beat and its nearest corresponding kinematic beat (peaks in motion velocity or acceleration):

$$\text{BA} = \frac{1}{|F^y|} \sum_{f_j^y \in F^y} \min_{f_i^x \in F^x} |f_j^y - f_i^x|$$

where:

- $F^y = \{f_j^y\}$ is the set of music beat timestamps (extracted from the audio using tools like Librosa)
- $F^x = \{f_i^x\}$ is the set of kinematic beat timestamps (detected from motion velocity peaks or acceleration peaks)
- $|f_j^y - f_i^x|$ is the temporal distance (in seconds or frames) between a music beat and a motion beat
- The inner `min` finds the closest motion beat to each music beat

Metrics - 4. Physical Foot Contact Score (PFC)

PFC evaluates the physical plausibility of generated motions by detecting and penalizing foot skating or foot sliding artifacts. Foot skating occurs when a foot that should be planted on the ground appears to slide unnaturally across the surface.

The PFC score measures how often predicted foot contact events (feet supposed to be on the ground) correspond to near-zero foot velocity:

$$\text{PFC} = \frac{1}{S-1} \sum_{i=1}^{S-1} \|\mathbf{v}_{\text{foot}}^i \cdot \mathbf{f}^i\|_2^2$$

where:

- S is the number of frames in the motion sequence
- $\mathbf{v}_{\text{foot}}^i = \text{FK}(\mathbf{R}_{i+1}) - \text{FK}(\mathbf{R}_i)$ is the velocity of foot joints between frames i and $i + 1$, computed via forward kinematics
- $\mathbf{f}^i \in \{0, 1\}^4$ is the binary foot contact label vector for frame i (indicating whether left foot, right foot, left toe, right toe are in contact)
- The Hadamard product $\mathbf{v}_{\text{foot}}^i \cdot \mathbf{f}^i$ masks the velocity, keeping only velocities of feet predicted to be in contact

Conclusion

Key Achievements:

ViMo successfully addresses a previously under-explored problem: generating diverse, realistic 3D human motions directly from casual videos with complex camera movements, occlusions, and editing—scenarios where traditional methods fail. The paper demonstrates that this goal is achievable through a simple but effective diffusion-based framework that conditions on 2D pose sequences rather than requiring explicit camera parameter estimation.

Demonstrated Impact:

The work opens three significant real-world applications: (1) efficient, large-scale 3D motion dataset construction from video sources (validated with 750 Chinese classical dance motions from 52 videos), (2) few-shot dance stylization enabling arbitrary style transfer with minimal examples, and (3) video-guided motion completion and editing with seamless stylistic blending.

Broader Implications:

ViMo reveals that motion generation diversity and flexibility can be substantially extended by leveraging abundant online video resources, moving beyond the constraints of expensive mocap systems and limited manually-curated datasets. This work positions video as a rich, underutilized data source for motion-related tasks.

Future Directions:

The authors emphasize this is a first attempt toward exploiting casual video content for motion generation and encourage future research to extend the framework toward more complex scenarios: semantic control, scene awareness, and multi-person motion generation. With continued development, the framework promises transformative potential for animation, interactive media, virtual characters, and the broader metaverse ecosystem.

For Implementation and More: [Refer](#)

Posterior of Diffusion Models I

We start from the posterior:

$$q(\mathbf{x}_{t-1} \mid \mathbf{x}_t, \mathbf{x}_0) = \frac{q(\mathbf{x}_t \mid \mathbf{x}_{t-1}, \mathbf{x}_0) q(\mathbf{x}_{t-1} \mid \mathbf{x}_0)}{q(\mathbf{x}_t \mid \mathbf{x}_0)}. \quad (7)$$

By the Markov property of the forward process:

$$q(\mathbf{x}_t \mid \mathbf{x}_{t-1}, \mathbf{x}_0) = q(\mathbf{x}_t \mid \mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t; \alpha_t \mathbf{x}_{t-1}, (1 - \alpha_t) \mathbf{I}). \quad (8)$$

Using Markov and Reparameterization trick for $\mathbf{x}_t$ and $\mathbf{x}_{t-1}$ respectively:

$$q(\mathbf{x}_{t-1} \mid \mathbf{x}_t, \mathbf{x}_0) = \frac{\mathcal{N}(\mathbf{x}_t; \sqrt{\alpha_t} \mathbf{x}_{t-1}, (1 - \alpha_t) \mathbf{I}) \mathcal{N}(\mathbf{x}_{t-1}; \sqrt{\bar{\alpha}_{t-1}} \mathbf{x}_0, (1 - \bar{\alpha}_{t-1}) \mathbf{I})}{\mathcal{N}(\mathbf{x}_t; \sqrt{\bar{\alpha}_t} \mathbf{x}_0, (1 - \bar{\alpha}_t) \mathbf{I})}. \quad (9)$$

From the definition ignoring the normalizing constants, we see that this expression has the form

Posterior of Diffusion Models II

$$\begin{aligned} \exp \bigg(& -\frac{1}{2} \frac{(\mathbf{x}_t - \sqrt{\alpha_t} \mathbf{x}_{t-1})^\top (\mathbf{x}_t - \sqrt{\alpha_t} \mathbf{x}_{t-1})}{1 - \alpha_t} \\ & -\frac{1}{2} \frac{(\mathbf{x}_{t-1} - \sqrt{\bar{\alpha}_{t-1}} \mathbf{x}_0)^\top (\mathbf{x}_{t-1} - \sqrt{\bar{\alpha}_{t-1}} \mathbf{x}_0)}{1 - \bar{\alpha}_{t-1}} \\ & +\frac{1}{2} \frac{(\mathbf{x}_t - \sqrt{\bar{\alpha}_t} \mathbf{x}_0)^\top (\mathbf{x}_t - \sqrt{\bar{\alpha}_t} \mathbf{x}_0)}{1 - \bar{\alpha}_t} \bigg). \end{aligned} \tag{10}$$

We will show that this expression matches

$$\mathcal{N}(\mathbf{x}_{t-1}; \boldsymbol{\mu}_q(\mathbf{x}_t, \mathbf{x}_0), \sigma_q^2(t) \mathbf{I}), \tag{11}$$

Posterior of Diffusion Models III

We can find $\sigma_q^2(t)$ by considering the coefficient of $-x_{t-1}^\top x_{t-1}$ in the exponent of (13). This coefficient is given by

$$\frac{1}{2} \frac{\alpha_t}{1 - \alpha_t} + \frac{1}{2} \frac{1}{1 - \bar{\alpha}_{t-1}} = \frac{1}{2} \frac{\alpha_t(1 - \bar{\alpha}_{t-1}) + 1 - \alpha_t}{(1 - \alpha_t)(1 - \bar{\alpha}_{t-1})} = \frac{1}{2} \left(\frac{1 - \bar{\alpha}_t}{(1 - \alpha_t)(1 - \bar{\alpha}_{t-1})} \right),$$

where we used $\alpha_t \bar{\alpha}_{t-1} = \bar{\alpha}_t$. Hence,

$$\sigma_q^2(t) = \frac{(1 - \alpha_t)(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t}. \quad (12)$$

Using the functional form of multivariate normal distribution, we can find $\mu_q(\mathbf{x}_t, \mathbf{x}_0)$ by considering the vector, say $\mathbf{v}$, such that $\mathbf{x}_{t-1}^\top \mathbf{v}$ is the cross-product in the exponent of (11). We see that

$$\frac{\mu_q(\mathbf{x}_t, \mathbf{x}_0)}{\sigma_q^2(t)} = \mathbf{v} = \frac{\sqrt{\alpha_t} \mathbf{x}_t}{1 - \alpha_t} + \frac{\sqrt{\bar{\alpha}_{t-1}} \mathbf{x}_0}{1 - \bar{\alpha}_{t-1}}. \quad (13)$$

Posterior of Diffusion Models IV

Hence, using (13),

$$\mu_q(\mathbf{x}_t, \mathbf{x}_0) = \frac{\sqrt{\alpha_t}(1 - \bar{\alpha}_{t-1})\mathbf{x}_t + \sqrt{\bar{\alpha}_{t-1}}(1 - \alpha_t)\mathbf{x}_0}{1 - \bar{\alpha}_t}. \quad (13)$$

We wish to compute a sample from the distribution in (12). This will allow us to perform the required transition along the backwards process. Our approach is to estimate the mean in (12) and then shift with an appropriate Gaussian in order to match the required variance.

If we know $\mathbf{x}_t$ and ϵ in (8), then we may write

$$\mathbf{x}_0 = \frac{\mathbf{x}_t - \sqrt{1 - \bar{\alpha}_t} \epsilon}{\sqrt{\bar{\alpha}_t}}. \quad (14)$$

Substituting this expression for $\mathbf{x}_0$ into (16), we see that the mean of $\mathbf{x}_{t-1}$, given $\mathbf{x}_t$ and $\mathbf{x}_0$, takes the form

Posterior of Diffusion Models V

$$\mu_q(\mathbf{x}_t, \mathbf{x}_0) = \frac{\sqrt{\alpha_t}(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t} \mathbf{x}_t + \frac{\sqrt{\bar{\alpha}_{t-1}}(1 - \alpha_t)}{1 - \bar{\alpha}_t \sqrt{\bar{\alpha}_t}} (\mathbf{x}_t - \sqrt{1 - \bar{\alpha}_t} \epsilon)$$

Substituting $\mathbf{x}_0 = (\mathbf{x}_t - \bar{\alpha}_t \epsilon) / \bar{\alpha}_t$:

$$\mu_q(\mathbf{x}_t, \mathbf{x}_0) = \frac{1}{\alpha_t} \left(\mathbf{x}_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon \right). \quad (15)$$

Here, ϵ is the noise that drove the transition from $\mathbf{x}_0$ to $\mathbf{x}_t$. We train a neural network to predict ϵ given t and $\mathbf{x}_t$, which can then be used to estimate the mean for the backward process.